# Design Document

## Basic Information

### Attempted Section

Testcase 1 and 2. Testcase 3 is not attempted.

### Assumptions

Before each execution of the test case begins, the system must start in a clean initial state. This means:

1. All queues initialized by the synchronizer must be empty.
2. All file semaphores must be completely removed.
3. The `cluster/` directory must not contain any files.

Point 1 has been addressed inside of the constructor of rabbitMQ, Point 2 and 3 is handled using the initial instructions in the `.vscode/task.json`

### Running this project

In the project, there is already a tasks.json file.  In VSCode, click Terminal → Run Task

To run Testcase 1 → click start BASIC

To run Testcase 2 → click start FULL (each in its own terminal)

## Architecture

**Coordinator**

The **Coordinator** starts with two exposed RPC functions: `Heartbeat()` and `GetServer()`. It maintains a routing table where servers can register themselves. When a client requests a connection, the coordinator uses this table to notify the client which server to connect to.

It can be launched using:

```
$./coordinator -p <portNum>
```

It provides three rpc endpoints.

1. Heartbeat: It enables the server and synchronizers to register themselves. At the first heartbeat, the coordinator stuff the information into the data structure. From the second heartbeat onwards, the coordinator will update the active status. The data structure maintained can be shown as below

```
struct BaseNode{
    std::string hostname;
    std::string port;
    std::string type;
    std::time_t last_heartbeat;
    bool missed_heartbeat;
    bool isActive();

};

struct zNode: public BaseNode {
    int serverID;

};

// node for synchronizers
struct syncNode: public BaseNode {
    int syncID;
};
```

2. GetServer: It offers two functions
    a. For the master server knowing the port number of the slave
    b. For the client to get the server port number to log into.
3. GetAllFollowerServers: It gets the current active synchronizers in the coordinator.

## Server

When the server is started using `./tsd`, it periodically sends heartbeat messages through the `Heartbeat()` RPC. This process registers the server with the coordinator and associates it with one of the three clusters. The server is initialized with the following command:

```
$./tsd -c <clusterId> -s <serverId>
-h <coordinatorIP> -k <coordinatorPort> -p <portNum>
```

After the client connects to it and issues an rpc, the server will not only does the operation specified, but also mirror the operations to the slave. For instance, when the client issues `Login()` rpc, it will not only login to the current user, but also mirrors the `Login()` operation to slave.

Besides the RPCs already provided, I also extended the **sns.proto** file to include two additional RPC endpoints.

1. `ReloadAllUsers` RPC

```
rpc ReloadAllUsers(Empty) returns (CoordConfirmation) {}

message Empty {}

message CoordConfirmation {
  bool success = 1;   // Indicates whether the operation succeeded
  string msg = 2;     // Optional status or error message
}
```

Whenever a synchronizer updates the `all_users.txt` file for a particular server, it can issue a `ReloadAllUsers` RPC call to notify the server. The sevrer then reloads the user information into `client_db` to ensure it remains consistent with the latest file state.

2. `ReloadAllFollowers` RPC

```
rpc ReloadAllFollowers(Empty) returns (CoordConfirmation) {}
```

Whenever a synchronizer modifies a `*_followers.txt` file, it triggers the `ReloadAllFollowers` RPC call. This instructs the server to reload follower relationships into `client_db`, ensuring follower data is up-to-date.

3. `ReloadAllTimelines` RPC

```
rpc ReloadAllTimelines(Empty) returns (CoordConfirmation){}
```

Whenever a synchronizer modifies a `*_following.txt` file, it triggers the `ReloadAllTimelines` RPC to notify the server that it should push the followee's new posts into the user's own feed.

Also, two helper functions, `read_file()` and `split_lines()` are also added. This helps the server to parse the all_users.txt and *_followers.txt file.

**Synchronizer**

When a user logs into the system, the information is only logged into their own server cluster. To keep all clusters consistent, we use a message-passing mechanism to exchange updates across clusters. In this assignment, RabbitMQ is used for this purpose.

The original rabbitMQ class has some problems, one noticeable problem is related to the `consumeMessage()`. The `amqp_basic_comsume()` will not consume the message directly from the queue specified. Instead, it will fetch from any queue that have data.

```
    std::string consumeMessage(const std::string &queueName, int timeout_ms
= 5000)
    {
        amqp_basic_consume(conn, channel, amqp_cstring_bytes(queueName.c
_str()),
                           amqp_empty_bytes, 0, 1, 0, amqp_empty_table);

        amqp_envelope_t envelope;
        amqp_maybe_release_buffers(conn);

        struct timeval timeout;
        timeout.tv_sec = timeout_ms / 1000;
        timeout.tv_usec = (timeout_ms % 1000) * 1000;

        amqp_rpc_reply_t res = amqp_consume_message(conn, &envelope, &tim
eout, 0);

        // ..

    }
```

In order to make the class more robust, several modifications are made

- Purging Queue at startup: making sure that the queue starts at a fresh state
- thread safety by using `amqpSafety`: This make sure that multiple process that uses `publishMessage` and `basicGet` will be serialized
- separate channels for publish and consume
    - `publish_channel` for `queue_declare`, `queue_purge`, `basic_publish`
    - `consume_channel` for `basic_get` + `amqp_read_message`

    This enables easier debugging since the responsibility of each channel is clear.

There are three types of information that needs to be synchronized across clusters,

- User information (e.g. all_users.txt)
- follower/following relationship (e.g. *_followers.txt)
- timeline updates (e.g. *_timelines)

Each type of information has dedicated queue for synchornization. Below is the queue that this MP provides, where synchID value ranges from 1 to 6.

```
synch<synchID>_users_queue
synch<synchID>_clients_relations_queue
synch<synchID>_timeline_queue
```

Here describes how the producer and consumer pushes to each queue.

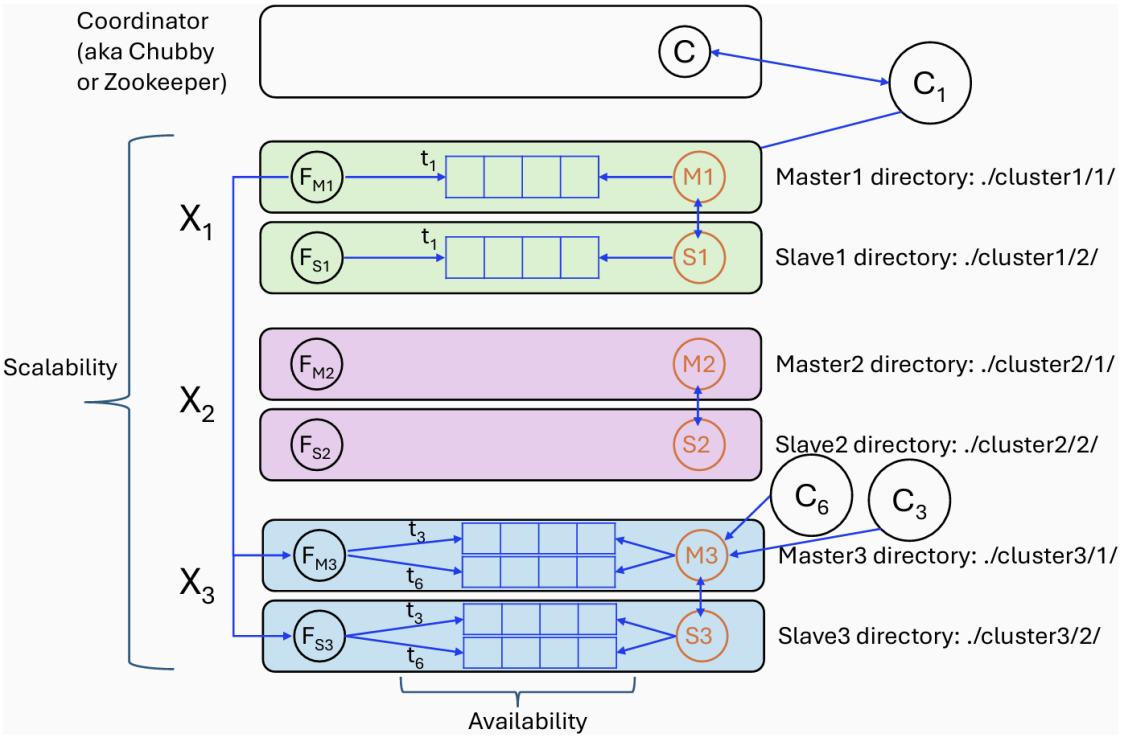
For synchronizing user information and follower/following relationship, each users will take their own `all_users.txt` and `*_followers.txt` and then publish to their own queue. The consumer will take the messages from all other queues except for itself.

For synchronizing timeline information, the master synchronizers publish the relevant portions of their `*_timeline.txt` files to the queues associated with each follower's master and slave clusters. Each synchronizer then consumes timeline messages from the queue corresponding to its own cluster.

It can be launched using the following command

```
 $./synchronizer-h <coordinatorIP>-k <coordinatorPort>-p <portNum>-i <syn
chronizerId>
```

The architecture follows the image below.




# Interaction between Different Parts

I will use Testcase 2 to explain how the client, server, coordinator, and synchronizer to interact.

1. Coordinator Intialization

The coordinator is started first. It waits for heartbeat RPCs from both servers and synchronizers:

```
./coordinator -p 9000
```

2. server and synchronizer initialization

Next, the servers and synchronizers are launched.

Each one sends an initial heartbeat RPC to register itself with the coordinator, and continues sending a heartbeat periodically to indicate that it is still alive.

```
./tsd -c 1 -s 1 -h localhost -k 9000 -p 10000
./tsd -c 1 -s 2 -h localhost -k 9000 -p 10001
```

```
./tsd -c 2 -s 1 -h localhost -k 9000 -p 20000
./tsd -c 2 -s 2 -h localhost -k 9000 -p 20001
./tsd -c 3 -s 1 -h localhost -k 9000 -p 30000
./tsd -c 3 -s 2 -h localhost -k 9000 -p 30001



./synchronizer -h localhost -k 9000 -p 9001 -i 1
./synchronizer -h localhost -k 9000 -p 9002 -i 2
./synchronizer -h localhost -k 9000 -p 9003 -i 3
./synchronizer -h localhost -k 9000 -p 9004 -i 4
./synchronizer -h localhost -k 9000 -p 9005 -i 5
./synchronizer -h localhost -k 9000 -p 9006 -i 6
```

3. Client initialization

The client is intialized and being routed to the coordinator to find the server port to log in.

```
./tsc -h localhost -k 9000 -u 1
./tsc -h localhost -k 9000 -u 2
./tsc -h localhost -k 9000 -u 3
```

4. Client Issues Commands

Once logged in, the client begins issuing commands.

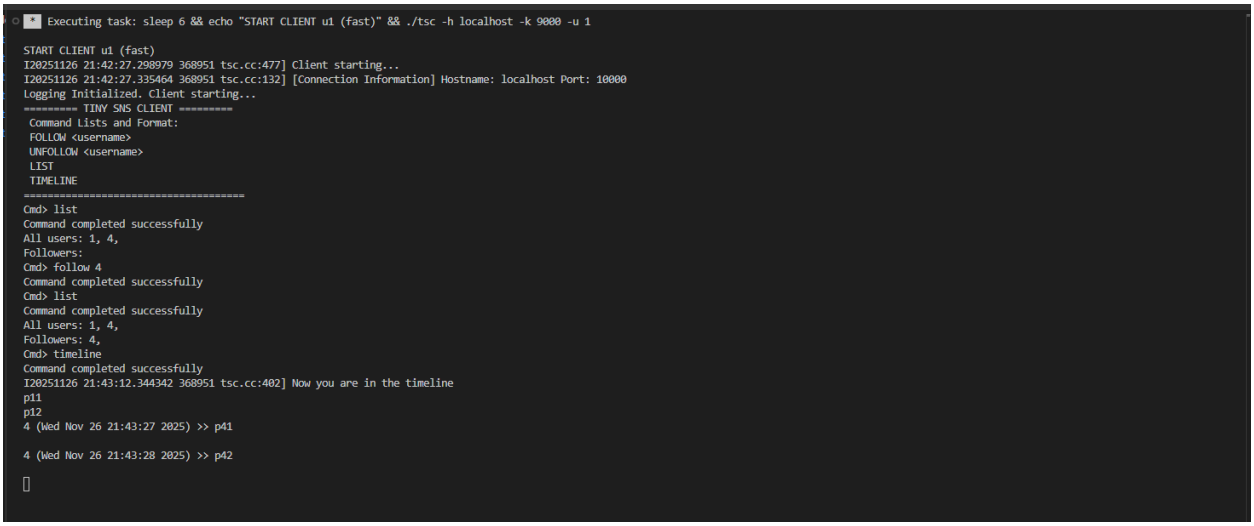
For example, when a user executes a **follow** command, the follower/following relationship is written to a file. Three things are happening

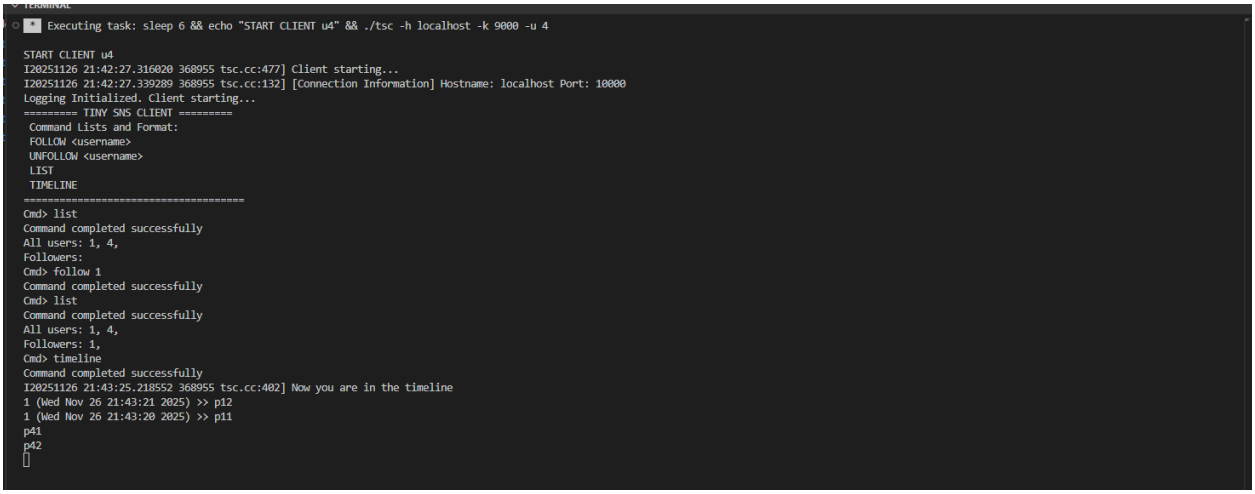
- The command is mirrored to the slave server
- The synchronizer propagates the updated file to other clusters using RabbitMQ
- After another cluster updates its files, it issues an RPC back to its server.

```
list
follow 2
```

# Result

## Testcase 1




## Testcase 2

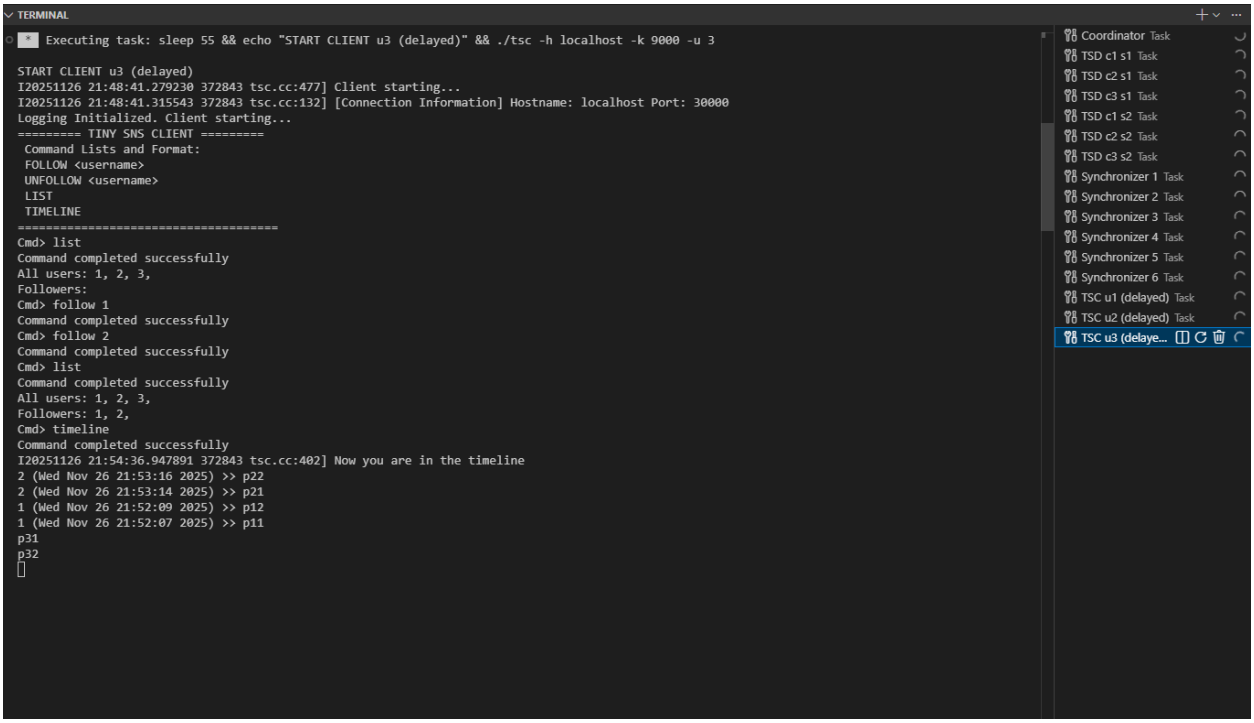
This tests is about whether the information can be propagated successfully between different clusters.

```
* Executing task: sleep 55 && echo "START CLIENT u1 (delayed)" && ./tsc -h localhost -k 9000 -u 1

START CLIENT u1 (delayed)
I20251126 21:48:41.278730 372839 tsc.cc:477] Client starting...
I20251126 21:48:41.312122 372839 tsc.cc:132] [Connection Information] Hostname: localhost Port: 10000
Logging Initialized. Client starting...
========= TINY SNS CLIENT =========
 Command Lists and Format:
 FOLLOW <username>
 UNFOLLOW <username>
 LIST
 TIMELINE
====================================
Cmd> list
Command completed successfully
All users: 1, 2, 3,
Followers:
Cmd> follow 2
Command completed successfully
Cmd> follow 3
Command completed successfully
Cmd> list
Command completed successfully
All users: 1, 2, 3,
Followers: 2, 3,
Cmd> timeline
Command completed successfully
I20251126 21:52:04.309360 372839 tsc.cc:402] Now you are in the timeline
p11
p12
2 (Wed Nov 26 21:53:14 2025) >> p21
2 (Wed Nov 26 21:53:16 2025) >> p22
3 (Wed Nov 26 21:54:39 2025) >> p31
3 (Wed Nov 26 21:54:40 2025) >> p32
```

```
* Executing task: sleep 55 && echo "START CLIENT u2 (delayed)" && ./tsc -h localhost -k 9000 -u 2

START CLIENT u2 (delayed)
I20251126 21:48:41.281697 372837 tsc.cc:477] Client starting...
I20251126 21:48:41.324538 372837 tsc.cc:132] [Connection Information] Hostname: localhost Port: 20000
Logging Initialized. Client starting...
========= TINY SNS CLIENT =========
 Command Lists and Format:
 FOLLOW <username>
 UNFOLLOW <username>
 LIST
 TIMELINE
====================================
Cmd> list
Command completed successfully
All users: 1, 2, 3,
Followers:
Cmd> follow 1
Command completed successfully
Cmd> follow 3
Command completed successfully
Cmd> list
Command completed successfully
All users: 1, 2, 3,
Followers: 1, 3,
Cmd> timeline
Command completed successfully
I20251126 21:53:11.344499 372837 tsc.cc:402] Now you are in the timeline
1 (Wed Nov 26 21:52:09 2025) >> p12
1 (Wed Nov 26 21:52:07 2025) >> p11
p21
p22
3 (Wed Nov 26 21:54:39 2025) >> p31
3 (Wed Nov 26 21:54:40 2025) >> p32
```


TERMINAL
* Executing task: sleep 55 && echo "START CLIENT u3 (delayed)" && ./tsc -h localhost -k 9000 -u 3
START CLIENT u3 (delayed)
I20251126 21:48:41.279230 372843 tsc.cc:477] Client starting...
I20251126 21:48:41.315543 372843 tsc.cc:132] [Connection Information] Hostname: localhost Port: 30000
Logging Initialized. Client starting...
========= TINY SNS CLIENT =========
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
====================================
Cmd> list
Command completed successfully
All users: 1, 2, 3,
Followers:
Cmd> follow 1
Command completed successfully
Cmd> follow 2
Command completed successfully
Cmd> list
Command completed successfully
All users: 1, 2, 3,
Followers: 1, 2,
Cmd> timeline
Command completed successfully
I20251126 21:54:36.947891 372843 tsc.cc:402] Now you are in the timeline
2 (Wed Nov 26 21:53:16 2025) >> p22
2 (Wed Nov 26 21:53:14 2025) >> p21
1 (Wed Nov 26 21:52:09 2025) >> p12
1 (Wed Nov 26 21:52:07 2025) >> p11
p31
p32
Coordinator Task
TSD c1 s1 Task
TSD c2 s1 Task
TSD c3 s1 Task
TSD c1 s2 Task
TSD c2 s2 Task
TSD c3 s2 Task
Synchronizer 1 Task
Synchronizer 2 Task
Synchronizer 3 Task
Synchronizer 4 Task
Synchronizer 5 Task
Synchronizer 6 Task
TSC u1 (delayed) Task
TSC u2 (delayed) Task
TSC u3 (delaye...